

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

«Программирование систем аутентификации и авторизации»

Выполнил:

Суханкулиев Мухаммет,
студент группы N3346



(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Архитектура и технологии.....	4
1.1 Схема базы данных.....	5
1.2 Схема взаимодействия при аутентификации.....	5
2 Реализация основных модулей.....	6
2.1 База данных и хранение пользователей (Задание 1)	6
2.2 Хеширование паролей (Задание 2)	6
2.3 Аутентификация и управление сессиями (Задания 3, 5)	6
2.4 Авторизация (Задание 4).....	7
3 Реализация дополнительных заданий (Задание 7)	8
3.1 Смена пароля.....	8
3.2 Многофакторная аутентификация (MFA).....	8
3.3 Управление ролями	8
4 Прототип и демонстрация работы	9
Заключение.....	12
Список использованных источников.....	13

ВВЕДЕНИЕ

Цель работы – получить практический опыт в разработке модульной и безопасной системы аутентификации и авторизации пользователей.

Для достижения поставленной цели необходимо решить следующие задачи:

- Разработать архитектуру системы с четким разделением ответственности между модулями.
- Реализовать механизм безопасного хранения учетных данных с использованием современного алгоритма хеширования (Argon2id).
- Создать модуль аутентификации, проверяющий подлинность пользователя по имени и паролю.
- Реализовать модуль управления сессиями на основе криптографически стойких токенов.
- Разработать модуль авторизации на основе ролей с поддержкой иерархии прав.
- Реализовать дополнительные функции: смену пароля, управление ролями и многофакторную аутентификацию (MFA).
- Создать два варианта пользовательского интерфейса: консольный (CLI) и веб-интерфейс (Web API + HTML/JS).

1 АРХИТЕКТУРА И ТЕХНОЛОГИИ

Для решения поставленных задач была спроектирована и реализована система на языке Python 3 с использованием многоуровневой архитектуры, обеспечивающей слабую связанность и высокую тестируемость компонентов.

Технологический стек:

- **Язык:** Python 3.12
- **База данных:** SQLite (для простоты развертывания).
- **ORM:** SQLAlchemy 2.0 в асинхронном режиме с драйвером `aiosqlite`.
- **Хеширование паролей:** `passlib` с алгоритмом `Argon2id`, который является победителем конкурса Password Hashing Competition и рекомендован OWASP.
- **Многофакторная аутентификация (MFA):** `pyotp` для реализации стандарта TOTP (Time-Based One-Time Password).
- **Веб-фреймворк:** FastAPI для создания Web API и `uvicorn` в качестве ASGI-сервера.
- **Валидация данных:** Pydantic для строгой типизации и валидации моделей данных.
- **Тестирование:** `pytest` и `pytest-asyncio` для написания модульных и интеграционных тестов.

Структура проекта:

Проект имеет четкую модульную структуру:

- **core/:** Модули, отвечающие за безопасность (`security.py`) и конфигурацию (`config.py`).
- **db/:** Модули для работы с базой данных, включая модели SQLAlchemy (`models.py`), настройку соединения (`database.py`) и слой доступа к данным (`repository.py`).
- **services/:** Слой бизнес-логики, содержащий сервисы для аутентификации, управления сессиями, ролями и MFA.
- **schemas/:** Схемы Pydantic, описывающие "контракты" данных между слоями.
- **frontend/:** Простой одностраничный веб-интерфейс на HTML и JavaScript.
- **main.py:** Точка входа для консольного приложения.

- **web_main.py**: Точка входа для веб-сервера FastAPI.

1.1 Схема базы данных

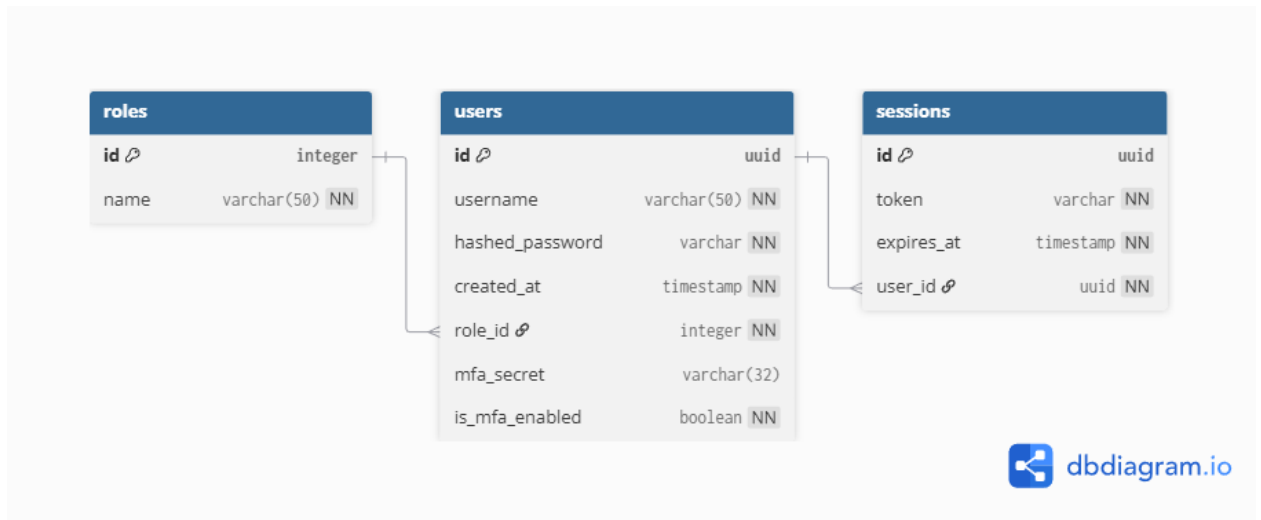


Рисунок 1 – Схема базы данных (ERD)

1.2 Схема взаимодействия при аутентификации

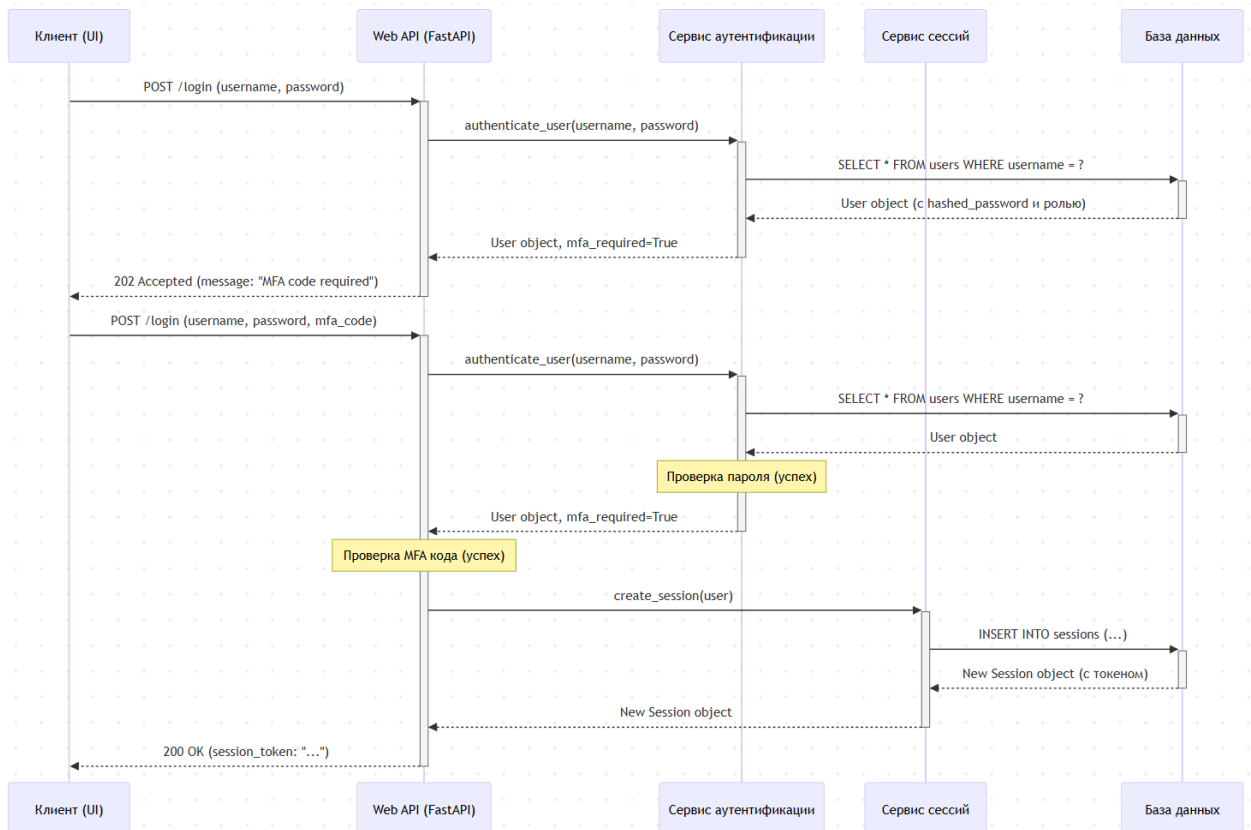


Рисунок 2 – Диаграмма последовательности для процесса входа с MFA

2 РЕАЛИЗАЦИЯ ОСНОВНЫХ МОДУЛЕЙ

2.1 База данных и хранение пользователей (Задание 1)

Была спроектирована схема базы данных, состоящая из трех таблиц:

1. **users**: хранит id (UUID), username, hashed_password, флаги MFA и внешний ключ на роль.
2. **roles**: справочник ролей (id, name).
3. **sessions**: таблица для хранения активных сессий (token, user_id, expires_at).

Использование SQLAlchemy ORM позволило абстрагироваться от прямого написания SQL-запросов, повысив безопасность (защита от SQL-инъекций) и читаемость кода.

2.2 Хеширование паролей (Задание 2)

В модуле `core/security.py` реализована логика хеширования. При регистрации пользователя его пароль не сохраняется в открытом виде. Вместо этого функция `hash_password` с помощью библиотеки `passlib` выполняет следующие действия:

1. Генерирует криптографически случайную соль (salt).
2. Применяет к паролю и соли алгоритм Argon2id с параметрами стойкости, заданными в конфигурации.
3. Сохраняет в базу данных одну строку, содержащую информацию об алгоритме, параметрах, соли и самом хеше.

Это обеспечивает защиту от атак перебором по словарю и по радужным таблицам.

2.3 Аутентификация и управление сессиями (Задания 3, 5)

Процесс аутентификации реализован в `AuthService`. При попытке входа:

1. Программа находит пользователя по имени в БД.
2. Функция `verify_password` извлекает соль и параметры из хеша в БД и применяет их к введенному паролю.
3. Сравниваются только итоговые хеши. Этот процесс устойчив к атакам по времени (timing attacks).

В случае успеха `SessionService` создает запись в таблице `sessions`, генерируя для нее криптографически стойкий токен (`secrets.token_hex(32)`). Этот токен имеет ограниченный срок жизни (1 час).

2.4 Авторизация (Задание 4)

Авторизация реализована на основе ролей.

- Метод `AuthService.authorize` принимает токен сессии и требуемую роль. Он находит пользователя по токену и сравнивает его роль с необходимой.
- Реализована простая иерархия прав: пользователь с ролью `admin` имеет доступ ко всем ресурсам, доступным роли `user`.

3 РЕАЛИЗАЦИЯ ДОПОЛНИТЕЛЬНЫХ ЗАДАНИЙ (ЗАДАНИЕ 7)

3.1 Смена пароля

В `AuthService` добавлен метод `change_password`. Он требует наличия активного токена сессии, проверяет корректность старого пароля и только после этого обновляет хеш в базе данных. Это защищает от несанкционированной смены пароля.

3.2 Многофакторная аутентификация (MFA)

Реализован второй фактор аутентификации на основе TOTP:

- В модель `User` добавлены поля `mfa_secret` и `is_mfa_enabled`.
- Создан `MFAService`, использующий `pyotp` для генерации секретных ключей и проверки одноразовых кодов.
- Администратор может через интерфейс включить MFA для любого пользователя. Система генерирует секретный ключ и URI для QR-кода.
- Процесс логина был доработан: если у пользователя включен MFA, после успешной проверки пароля система запрашивает второй фактор — 6-значный код из приложения-аутентификатора. Сессия создается только после успешной проверки обоих факторов.

3.3 Управление ролями

Для администраторов реализован следующий функционал:

- Просмотр списка всех существующих ролей.
- Создание новой роли.
- Назначение существующей роли любому пользователю в системе.

Вся эта логика инкапсулирована в `RoleService` и защищена проверкой прав администратора в слое представления (API).

4 ПРОТОТИП И ДЕМОНСТРАЦИЯ РАБОТЫ

Было создано два интерфейса для взаимодействия с системой:

1. **Консольный интерфейс** (`main.py`): Позволяет выполнять все реализованные функции через текстовые команды. Удобен для быстрой проверки и отладки ядра системы.
2. **Веб-интерфейс** (`web_main.py` + `frontend/index.html`):
 - **Бэкенд:** FastAPI предоставляет RESTful API со всеми функциями системы. Эндпоинты защищены проверкой токена в заголовке `X-Auth-Token`. Админские эндпоинты дополнительно проверяют роль пользователя. API документирован с помощью Swagger UI.
 - **Фронтенд:** Простая одностраничная HTML-страница с JavaScript, которая отправляет запросы к API и отображает результаты, предоставляя пользователю интерактивный опыт.

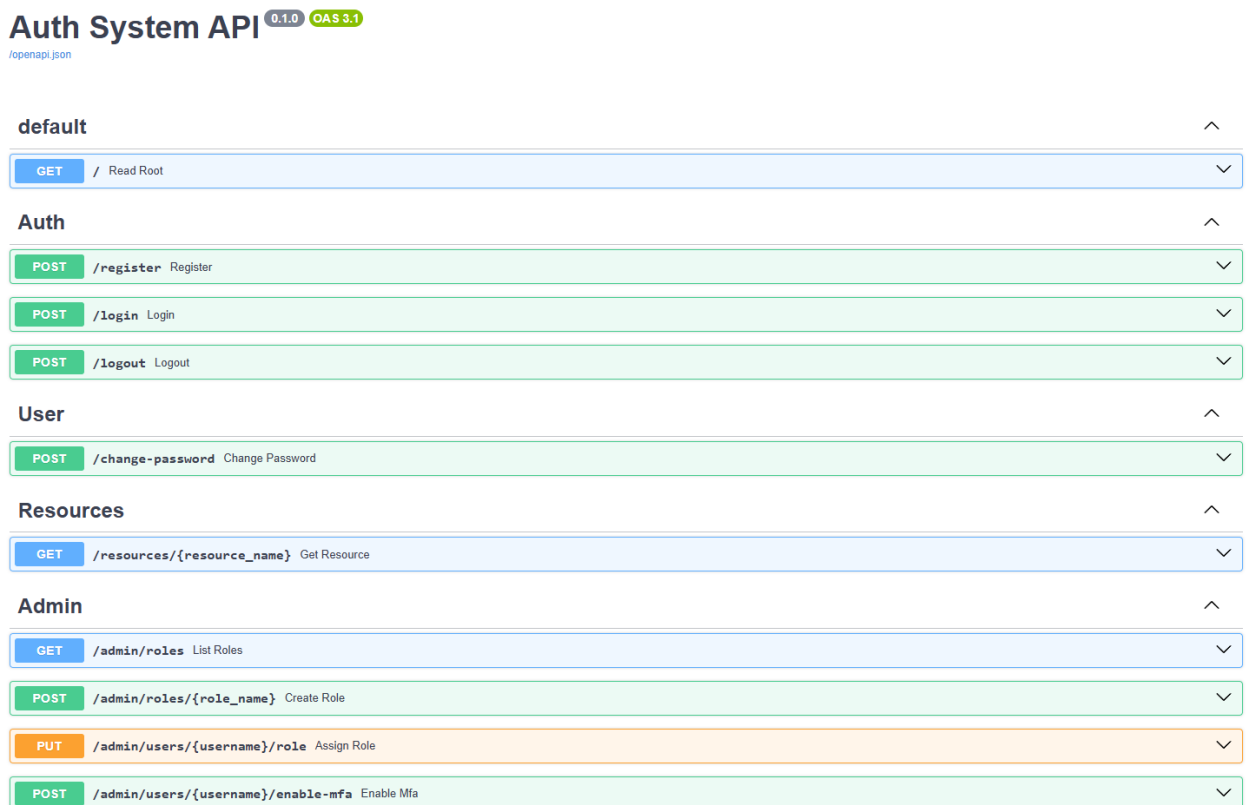


Рисунок 3 – Интерактивная документация API, сгенерированная FastAPI

Auth System API Client

Добро пожаловать!

Токен сессии сохранен.

Действия пользователя

Старый пароль

Новый пароль

Сменить пароль

Общий ресурс

Получить доступ к ресурсу

Панель администратора

Показать все роли

Название новой роли

Создать роль

Имя пользователя для MFA

Включить MFA

Имя пользователя

Новая роль

Назначить роль

Выйти

Статус: 200

```
{
  "session_token":
    "38fc7cc5b912d61be88a72ea8b905ab3e1fcf32978ab1a1a0a799463b1c
    da6b8",
  "message": null
}
```

Рисунок 4 – Главный экран веб-интерфейса после аутентификации администратора



Рисунок 5 – Процесс активации многофакторной аутентификации (MFA)

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно спроектирована и реализована комплексная система аутентификации и авторизации. Были изучены и применены на практике ключевые принципы безопасности: безопасное хранение паролей с использованием Argon2id, управление сессиями через токены, авторизация на основе ролей и реализация многофакторной аутентификации (TOTP).

Разделение системы на логические слои (представление, сервисы, репозитории) позволило создать гибкое и переиспользуемое ядро, к которому были успешно подключены два разных типа интерфейса — консольный и веб. Это подтверждает правильность выбранного архитектурного подхода. Цели лабораторной работы достигнуты в полном объеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ищенко А. П. Технологии и методы программирования – Лекция 5: материалы лекции / А. П. Ищенко. – Санкт-Петербург : Университет ИТМО, 2025. – Текст : непосредственный.
2. PEP 8 -- Style Guide for Python Code : стандарт. – Текст : электронный // Python Software Foundation. – 2013. – URL: <https://peps.python.org/pep-0008/>
3. FastAPI : официальная документация. – Текст : электронный // Sebastián Ramírez. – 2024. – URL: <https://fastapi.tiangolo.com/>
4. SQLAlchemy 2.0 ORM : официальная документация. – Текст : электронный // SQLAlchemy authors. – 2024. – URL: <https://docs.sqlalchemy.org/en/20/>
5. Passlib : официальная документация. – Текст : электронный // Assurance Technologies. – 2022. – URL: <https://passlib.readthedocs.io/>
6. OWASP Password Storage Cheat Sheet : рекомендации по безопасности. – Текст : электронный // Open Web Application Security Project. – 2023. – URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html